

Find the Missing Connections

Complete Step-by-Step Implementation Prompts

18 prompts in implementation order. Use them one by one in your coding AI.

Prompt 1 — Project Setup

Build the foundation of a web application called "Find the Missing Connections".

The project analyzes articles and finds:

1. People and places mentioned in articles that do not have their own article.
2. Connections that exist in one direction but not in the reverse direction.

Use only free/open technologies.

Technology:

- Frontend: React
- Backend: Python FastAPI
- Database: PostgreSQL
- Article source: Wikipedia / MediaWiki API
- Connection visualization: Cytoscape.js

Create this structure:

```
frontend/  
backend/  
database/  
README.md
```

Prepare the frontend for:

- Article search
- Article analysis
- Missing connections
- One-way connections
- Connection map

Prepare the backend for:

- Article retrieval
- Link extraction
- Article existence checking
- Missing connection detection
- One-way connection detection
- Connection map generation

Create a clean homepage titled:

"Find the Missing Connections"

Description:

"Discover people and places mentioned in articles that don't have their own article yet, or connections that only work one way."

Add an article search field and Analyze button.

Do not implement advanced features yet.

Do not add authentication, recommendations, analytics, chatbots, scoring, payments, or unrelated features.

Prompt 2 — Article Search

Implement the Article Search functionality for the existing "Find the Missing Connections" project.

Use the Wikipedia / MediaWiki API.

When the user enters an article name:

1. Send the search request to the backend.
2. Backend queries the MediaWiki API.

3. Find the matching article.
4. Retrieve:
 - Article title
 - Article URL
 - Article content or extract
 - Page ID
5. Return the information to the frontend.

Frontend should display:

Article title

Article URL

Short article extract

“Analyze Article” button

Handle:

- Article found
- Article not found
- Empty search
- API errors
- Network errors

Do not add unrelated functionality.

Do not add AI features yet.

Prompt 3 — Retrieve Article Links

Implement the article link extraction functionality.

For the selected article, retrieve the links contained inside the article using the MediaWiki API.

For every link, collect:

- Link title
- Link target
- Source article
- Whether it is an internal article link

Ignore:

- External websites
- Images
- Files
- Categories
- Navigation elements
- Non-article resources

Return the extracted links from the backend to the frontend.

Display them under:

“Article Connections”

Example:

Albert Einstein

Connections:

- Germany
- Switzerland
- Max Planck
- University of Zurich
- Physics

Do not add AI or recommendation features.

Prompt 4 — Check Whether Articles Exist

Implement article existence checking.

For every extracted person/place/article link, determine whether the target has an existing Wikipedia article.

For every connection, classify it as:

EXISTS

or

MISSING

Use the MediaWiki API rather than scraping Wikipedia HTML.

Handle:

- Existing pages
- Missing pages
- Redirects
- Invalid pages
- API errors

Return structured results to the frontend.

Example:

Article:

Albert Einstein

Connections:

Germany – EXISTS

Switzerland – EXISTS

Max Planck – EXISTS

Example Person – MISSING

Display missing connections clearly.

Do not implement one-way connection detection yet.

Prompt 5 — Missing Connections

Implement the main Find the Missing Connections functionality.

After checking all article links, create a dedicated section:

“Missing Connections”

Display every person or place mentioned/linked from the analyzed article that does not have an existing Wikipedia article.

For every missing connection show:

- Name
- Type if available
- Source article
- Status: MISSING

Example:

Missing Connections

1. Person A
Status: Missing article
2. Place B
Status: Missing article
3. Person C
Status: Missing article

Clearly distinguish missing connections from existing connections.

Do not add scoring, ranking, recommendations, or unrelated features.

Prompt 6 — Detect People and Places

Improve the connection extraction so the system identifies names of people and places mentioned in the article.

For each detected entity, store:

- Name
- Entity type:
 - Person
 - Place
 - Other
- Source article
- Wikipedia link if available
- Article existence status

The primary focus must remain on:

People

Places

Do not introduce unrelated entity categories as major project features.

Use reliable article/link information from the Mediawiki API where possible.

Display the detected people and places separately:

People

Places

Missing People

Missing Places

Prompt 7 — One-Way Connections

Implement one-way connection detection.

For every existing article connection:

Article A → Article B

Check whether Article B links back to Article A.

Classify the relationship as:

1. BIDIRECTIONAL
 - A → B
 - B → A
2. ONE-WAY
 - A → B
 - B does not link to A

Display one-way connections in a dedicated section:

“One-Way Connections”

Example:

Albert Einstein → Max Planck

Reverse connection:
Not found

Status:
ONE-WAY

Do not modify article data.
Do not add recommendations or AI explanations.

Prompt 8 — Connection Map Data

Create the backend data structure required to visualize article connections.

Represent the information as a graph:

Nodes:

- Analyzed article
- Existing linked articles
- Missing people
- Missing places

Edges:

- Source article → target article/entity

Each node should contain:

- ID
- Name
- Type
- Exists status

Each edge should contain:

- Source
- Target
- Connection status

Connection statuses:

- EXISTS
- MISSING
- ONE-WAY
- BIDIRECTIONAL

Return the graph data through a FastAPI endpoint.

Do not create the visual graph yet.

Prompt 9 — Interactive Connection Map

Implement the Connection Map using Cytoscape.js.

Display the analyzed article as the central node.

Show connected people, places, and articles around it.

The map must visually distinguish:

- Existing connections
- Missing connections
- One-way connections

Users should be able to:

- Zoom
- Pan
- Select a node
- View the node name
- View whether the article exists

Example structure:

Article A

■■■ Person A

■■■ Place A

■■■ Article B

■■■ Person B — MISSING

Keep the graph focused on the analyzed article and its discovered connections.

Do not add unrelated graph features.

Prompt 10 — Missing Connection Highlighting

Improve the Connection Map so missing connections are clearly highlighted.

A missing person or place must be visually distinguishable from existing articles.

When the user selects a missing node, show:

Name

Type

Source article

Status:

“MISSING ARTICLE”

For existing nodes show:

Name

Type

Status:

“ARTICLE EXISTS”

For one-way connections show:

Source

Target

Status:

“ONE-WAY CONNECTION”

Do not introduce scoring or priority systems.

Prompt 11 — Article Analysis Page

Create the main Article Analysis page.

After the user selects an article, display:

Article title

Article URL

Article extract

People

Places

Article Connections

Missing Connections

One-Way Connections

Connection Map

Organize these sections clearly so the user can understand the relationships.

The page should remain focused entirely on the “Find the Missing Connections” project.

Do not add analytics dashboards, recommendations, authentication, or AI assistant features.

Prompt 12 — Database Storage

Connect the application to PostgreSQL.

Store only the information required for the Find the Missing Connections project.

Create tables for:

Articles

- id
- page_id
- title
- url

Connections

- id
- source_article_id
- target_name
- target_article_id if available
- target_type
- exists_status

Connection Relationships

- id
- source_article_id
- target_article_id
- relationship_status

Store:

- Existing connections
- Missing connections
- One-way connections
- Bidirectional connections

Do not store unnecessary user data or unrelated information.

Make sure duplicate connections are handled correctly.

Prompt 13 — Complete Analysis Workflow

Connect all implemented components into one complete workflow.

When a user enters an article:

1. Search for the article.
2. Retrieve the article.
3. Extract its internal links.
4. Identify people and places where possible.
5. Check whether each linked entity has an article.
6. Mark missing connections.
7. Check reverse links for existing article connections.
8. Identify one-way connections.
9. Generate connection graph data.
10. Display the connection map.

The complete flow must work from the single “Analyze Article” action.

Handle loading states and errors properly.

Do not add features outside the project requirements.

Prompt 14 — Error Handling

Add robust error handling to the existing Find the Missing Connections application.

Handle:

- Empty article name
- Article not found
- Wikipedia API unavailable
- Network failure
- Invalid API response
- Missing article data
- Database failure
- Reverse-link lookup failure
- Graph data failure

Show clear user-friendly messages.

Examples:

"Please enter an article name."

"Article not found."

"Unable to retrieve article connections."

"Unable to check the reverse connection."

Do not expose technical stack traces to normal users.

Do not add unrelated functionality.

Prompt 15 — Loading States

Add loading states for the article analysis process.

Show appropriate loading indicators while:

- Searching for an article
- Retrieving article content
- Extracting links
- Checking article existence
- Checking reverse connections
- Building the connection map

The user should understand that analysis is still running.

Do not add animations that distract from the application.

Do not add unrelated UI.

Prompt 16 — Final UI Polish

Polish the existing Find the Missing Connections interface without changing its functionality.

Keep the interface simple, clean, and research-oriented.

Improve:

- Typography
- Spacing
- Buttons
- Input fields
- Article sections
- Missing connection visibility
- One-way connection visibility
- Connection map layout
- Responsive behavior

The primary navigation should contain only:

- Search
- Analysis

The analysis page should contain:

- Article
- People
- Places
- Connections
- Missing Connections
- One-Way Connections
- Connection Map

Do not add:

- Authentication
- Payments
- Recommendations
- Analytics
- AI chatbot
- Social features
- User profiles
- Notifications
- Other unrelated features.

Prompt 17 — Testing

Test the complete Find the Missing Connections application.

Test the following:

Article Search

- Valid article
- Invalid article
- Empty input

Link Extraction

- Article with many links
- Article with no relevant links
- Internal links
- External links

Missing Connections

- Existing article
- Missing article
- Missing person
- Missing place

One-Way Connections

- $A \rightarrow B$ and $B \rightarrow A$
- $A \rightarrow B$ but B does not link to A

Connection Map

- Existing nodes
- Missing nodes
- One-way edges
- Multiple connections

API

- Successful response
- Failed response
- Timeout
- Invalid response

Database

- Insert article
- Insert connection
- Retrieve connections
- Prevent duplicate records

Fix all errors found during testing.

Do not introduce new functionality during testing.

Prompt 18 — Final Project Verification

Perform a final verification of the Find the Missing Connections project.

Confirm that the application can:

1. Accept an article name.
2. Retrieve the article.
3. Extract names and links.
4. Identify people and places.
5. Check whether their articles exist.
6. Identify missing connections.
7. Check reverse article connections.
8. Identify one-way connections.
9. Generate a connection map.
10. Highlight missing connections.

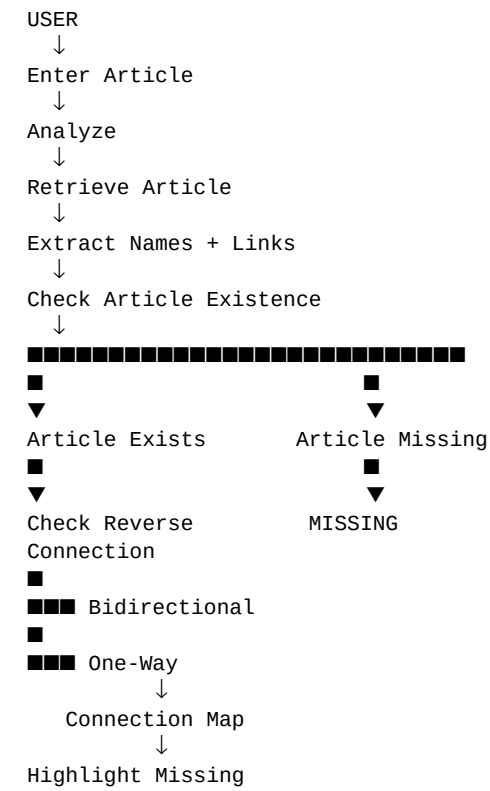
Verify that:

- Frontend works.
- Backend works.
- PostgreSQL works.
- Mediawiki API integration works.
- Cytoscape.js graph works.
- Frontend/backend communication works.
- Error handling works.
- The application works locally.
- README contains setup instructions.

Remove any code or functionality that is unrelated to the project requirements.

Do not add any new features.

Final Project Flow



Advanced Challenges

- Retrieval-Augmented Generation (RAG)
- Semantic Search
- AI Research Assistant
- Interactive Knowledge Graph
- Article Recommendation Engine
- Topic Classification
- Educational Learning Assistant